

RoK4 ADAPT Action/Actuator 제어 구조 문서

작성일: 2026-07-16
대상 저장소: /home/rcclab/rok4_lab
기준 환경: Isaac Lab v2.3.2, Isaac Sim 5.1.0, env_isaacclab

문서 목적

이 문서는 RoK4의 policy action이 ADAPT actuator position target을 거쳐 PhysX joint effort가 되기까지의 전체 제어 경로를 설명한다. 특히 다음처럼 코드만 보면 혼동하기 쉬운 부분을 구분한다.

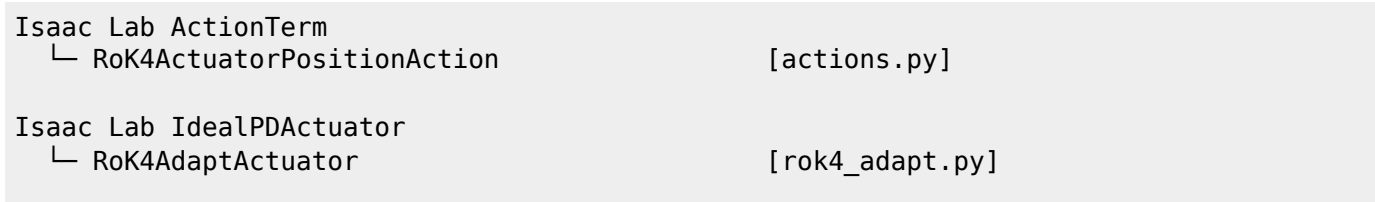
- actions.py 와 rok4_adapt.py 의 관계는 상속이 아니라 runtime 객체 참조다.
- policy는 actuator torque가 아니라 normalized actuator position offset을 출력한다.
- actions.py 가 기록하는 q_target 은 PhysX position drive 명령이 아니라 explicit actuator의 입력이다.
- compute() 의 joint_pos 와 joint_vel 은 action이 아니라 PhysX가 계산한 현재 robot state다.
- 현재 velocity target은 사용되지 않는 변수가 아니라 값이 0인 damping 목표다.
- control_action.joint_velocities 는 표준 runtime 입력에서는 None 이 아니라 zero tensor다.
- compute() 출력에서 position/velocity를 None 으로 만드는 것은 PhysX drive를 끄기 위한 별도 단계다.

관련 파일과 역할

파일	주요 역할
assets/robots/rok4.py	joint/actuator 순서, ADAPT 링크 길이, gain, action scale, torque/velocity limit과 ROK4_TRAIN_CFG 정의
assets/robots/rok4_adapt.py	ADAPT 행렬 변환과 actuator-space explicit PD torque 계산
velocity/mdp/actions.py	policy raw action을 psi_target 과 mapped q_target 으로 변환
velocity/mdp/observations.py	현재 joint state를 actuator-space observation으로 변환
velocity/mdp/rewards.py	actuator torque/velocity/acceleration, limit, action smoothness reward 계산
config/rok4/flat_env_cfg.py	action/observation/reward term을 task에 조립하고 timing과 weight 설정

Class 상속과 객체 참조 관계

RoK4ActuatorPositionAction 과 RoK4AdaptActuator 사이에는 Python 상속 관계가 없다. 두 class는 서로 다른 Isaac Lab base class를 상속하고, action term이 robot asset에 이미 생성된 actuator instance를 찾아 그 안의 transmission 객체를 참조한다.



└ contains RoK4AdaptTransmission

Runtime reference

```
RoK4ActuatorPositionAction.__init__()  
-> actuator = robot.actuators["body"]  
-> type check: isinstance(actuator, RoK4AdaptActuator)  
-> self._transmission = actuator.transmission
```

따라서 action과 actuator는 같은 RoK4AdaptTransmission instance를 공유한다. Action target과 actuator PD가 서로 다른 ADAPT 행렬을 별도로 만들지 않으므로 링크 길이나 행렬 정의가 어긋나지 않는다.

좌표 순서와 변환 대상

모든 policy/ADAPT 계산은 rok4.py 의 ROK4_JOINT_ORDER 를 기준으로 한다.

```
index 0  L_Hip_Yaw_Joint      direct  
index 1  L_Hip_Roll_Joint    direct  
index 2  L_Hip_Pitch_Joint    }  
index 3  L_Knee_Pitch_Joint    } left ADAPT block  
index 4  L_Ankle_Pitch_Joint   }  
index 5  L_Ankle_Roll_Joint    }  
  
index 6  R_Hip_Yaw_Joint      direct  
index 7  R_Hip_Roll_Joint    direct  
index 8  R_Hip_Pitch_Joint    }  
index 9  R_Knee_Pitch_Joint    } right ADAPT block  
index 10 R_Ankle_Pitch_Joint   }  
index 11 R_Ankle_Roll_Joint    }  
  
index 12 Torso_Yaw_Joint      direct
```

RoK4AdaptTransmission.COUPLED_SLICES 는 slice(2, 6) 과 slice(8, 12) 이다. _map_coupled() 는 13차원 입력을 먼저 clone한 뒤 이 두 4차원 구간만 행렬 변환한다. Index 0,1,6,7,12 는 clone된 값이 그대로 남으므로 direct-drive coordinate로 통과한다.

ADAPT position 행렬

한쪽 다리의 coupled coordinate를 column vector로 표현하면 다음 관계를 사용한다.

```
q = J psi  
psi = J^-1 q  
  
J = [ 0.5,    0.5,    0,    0 ]  
    [ 0.5,   -0.5,    0,    0 ]  
    [-0.5,    0.5,   0.5,   0.5 ]  
    [ 0,      0, -beta/alpha, beta/alpha ]
```

현재 alpha=0.09845 m, beta=0.06 m 이며 ratio=beta/alpha 다. PyTorch state는 [num_envs, 13] 의 row-vector batch이므로 column-vector 수식과 transpose 방향이 달라진다.

함수	Column-vector 수식	코드의 row-vector 연산
actuator_to_joint_position	$q = J \psi$	<code>psi_row @ J.T</code>
joint_to_actuator_position	$\psi = J^{-1} q$	<code>q_row @ J^-1.T</code>

joint_to_actuator_position(values) 의 실제 helper 동작을 반복문 두 번으로 펼치면 다음과 같다.

```
mapped = values.clone()
mapped[..., 2:6] = values[..., 2:6] @ q_j_psi_inv.T
mapped[..., 8:12] = values[..., 8:12] @ q_j_psi_inv.T
return mapped
```

한쪽 다리 joint block을 [q_hp, q_knee, q_ap, q_ar] 로 놓으면 actuator position은 다음과 같다.

```
psi_1 = q_hp + q_knee
psi_2 = q_hp - q_knee
psi_3 = q_knee + q_ap - alpha/(2 beta) * q_ar
psi_4 = q_knee + q_ap + alpha/(2 beta) * q_ar
```

현재 $\alpha/(2 \beta)$ 는 약 0.8204 다. 따라서 ADAPT block 안에서는 joint 하나가 actuator 하나에만 대응하지 않고 여러 actuator coordinate에 함께 반영된다.

ADAPT torque 행렬

Position과 torque는 같은 방향의 행렬을 사용하지 않는다. Virtual-work 관계로 다음 식을 사용한다.

```
tau_psi = J.T tau_q
tau_q = J-T tau_psi
```

Row-vector batch 코드에서는 다음과 같다.

함수	물리 수식	코드 연산
actuator_to_joint_torque	$\tau_q = J^{-T} \tau_{\psi}$	$\tau_{\psi_row} @ J^{-1}$
joint_to_actuator_torque	$\tau_{\psi} = J.T \tau_q$	$\tau_{q_row} @ J$

Policy step의 action 처리

Policy는 100 Hz로 13차원 raw actuator action을 출력한다. process_actions() 는 policy step마다 한 번 다음 순서로 target을 준비한다.

```
policy output [num_envs, 13]
-> raw_action을 [-1, 1]로 clip
-> actuator_offset = raw_action * ROK4_ACTUATOR_ACTION_SCALE
-> psi_default = J-1 q_default
-> psi_target = psi_default + actuator_offset
-> q_target = J psi_target
-> processed_actions에 q_target 보관
```

apply_actions() 는 이 q_target 을 Articulation.set_joint_position_target() 으로 기록한다. 이 함수는 즉시 PhysX position drive를 실행하지 않고 ArticulationData.joint_pos_target buffer만 갱신한다.

학습 시작 시 ADAPT 진단 출력

RoK4ActuatorPositionAction 은 초기화할 때 전체 environment를 반복 출력하지 않고 첫 environment의 공통 default pose를 사용해 한 번 진단한다. 먼저 q_J_psi, q_J_psi_inv, q_J_psi_T, q_J_psi_invT 네 행렬을 출력하고 다음 최대 오차를 검사한다. Isaac Lab RSL-RL 학습은 CUDA TF32 행렬곱을 활성화하므로 작은

4x4 FP32 검증 행렬곱에도 약 $2.44e-4$ 의 반올림 오차가 생길 수 있다. 이는 ADAPT 행렬 정의 오류가 아니므로 행렬의 구조적 관계와 기준 왕복 변환은 CPU FP64로 검사한다.

```
max|J @ J^-1 - I|
max|J^-1 @ J - I|
max|stored J^T - J^T|
max|stored J^-T - J^-T|
max|J(J^-1 q_default) - q_default| (CPU FP64 reference)
max|psi_default runtime - psi_default FP64|
max|q_default runtime round trip - q_default|
```

CPU FP64 구조 검사는 $1e-5$ 를, 실제 CUDA FP32/TF32 mapping과 왕복 변환은 $5e-4$ rad 를 넘으면 학습 전에 RuntimeError 를 발생시킨다. 검사를 통과하면 canonical 13-coordinate 순서로 q_default (13x1) -> psi_default (13x1) 변환 표를 출력한다. 따라서 _R0K4_INIT_JOINT_POS 가 올바른 순서와 ADAPT 행렬로 _default_actuator_pos 에 들어갔는지 training log에서 바로 확인할 수 있다.

q_target 을 만드는 이유

Policy action은 actuator 좌표인데도 actions.py 가 다시 q_target 을 만드는 이유는 Isaac Lab actuator interface가 articulation joint target buffer를 통해 custom actuator에 target을 전달하기 때문이다.

```
actions.py
psi_target -> q_target
-> joint_pos_target buffer
-> Articulation._apply_actuator_model()
-> RoK4AdaptActuator.compute()
-> q_target -> psi_target
```

겉으로는 왕복 변환처럼 보이지만 선형이고 invertible한 같은 J 를 공유하므로 원래 psi_target 이 복원된다. 이 q_target 은 explicit actuator 입력이며 PhysX joint position drive에 직접 전달되지 않는다.

Physics step의 actuator 계산

Physics는 500 Hz로 실행되고 decimation=5 이므로 하나의 policy target을 유지한 채 actuator PD를 5번 다시 계산한다. 각 physics step에서 Isaac Lab Articulation._apply_actuator_model() 이 target buffer와 현재 state를 모아 RoK4AdaptActuator.compute() 를 호출한다.

```
Articulation target buffers
joint_pos_target
joint_vel_target
joint_effort_target
+
Current PhysX state
joint_pos
joint_vel
|
v
RoK4AdaptActuator.compute()
-> canonical ADAPT 순서로 reorder
-> q, q_target      -> psi, psi_target
-> q_dot            -> psi_dot
-> q_dot_target     -> psi_dot_target
-> tau_psi_requested = Kp(psi_target-psi) + Kd(psi_dot_target-psi_dot) + tau_ff
```

```
-> actuator torque limit으로 tau_psi_applied clip
-> tau_q = J^-T tau_psi_applied
-> USD model order로 복원
-> PhysX joint effort 반환
```

compute() 입력 변수의 출처

compute() 입력	현재 RoK4 task의 값	출처와 의미
control_action.joint_positions	q_target tensor	actions.py 가 joint_pos_target buffer에 기록한 목표 joint position
control_action.joint_velocities	zero tensor	Isaac Lab이 초기화한 joint_vel_target buffer; 현재 action term은 이 buffer를 수정하지 않음
control_action.joint_efforts	zero tensor	Isaac Lab이 초기화한 feedforward joint_effort_target buffer
joint_pos	현재 q tensor	PhysX simulation의 현재 joint position state; action 값이 아님
joint_vel	현재 q_dot tensor	PhysX simulation의 현재 joint velocity state; action 값이 아님

모든 tensor의 마지막 차원은 actuator group의 13개 joint이며 environment batch가 앞에 붙는다.

현재 velocity target과 damping

현재 task는 position action만 사용하므로 nonzero velocity target을 명령하지 않는다. 그러나 actuator_vel_target 변수가 사용되지 않는 것은 아니다. 실제 값이 0인 목표 속도로 damping 항에 사용된다.

```
joint_vel_target = 0
-> actuator_vel_target = J^-1 * 0 = 0

tau_psi
= Kp(psi_target - psi)
  + Kd(actuator_vel_target - actuator_vel)
  + tau_ff

= Kp(psi_target - psi) - Kd psi_dot
```

따라서 현재 actuator_vel_target 은 zero-velocity damping reference다. 향후 별도의 velocity target을 쓰면 같은 코드가 nonzero target을 처리할 수 있다.

None 분기의 실제 의미

Isaac Lab Articulation 은 초기화할 때 position, velocity, effort target buffer를 모두 zero tensor로 할당한다. 따라서 현재 표준 RoK4 runtime에서 compute() 로 들어오는 control_action.joint_velocities 와 control_action.joint_efforts 는 None 이 아니라 zero tensor다.

```

if control_action.joint_velocities is None:
    actuator_vel_target = torch.zeros_like(actuator_vel)
else:
    # 현재 runtime은 이 경로이며 입력 tensor 값이 모두 0이다.
    actuator_vel_target = joint_to_actuator_velocity(joint_vel_target)

```

None 입력 분기는 custom actuator를 별도로 호출하거나 target이 생략된 ArticulationActions 를 받을 때를 위한 방어 코드다. 현재 ManagerBasedEnv의 정상 호출에서는 실행되지 않는다.

반면 compute() 마지막의 다음 코드는 출력 단계의 의미가 다르다.

```

control_action.joint_efforts = applied_joint_effort
control_action.joint_positions = None
control_action.joint_velocities = None

```

입력 target을 계산에 사용한 뒤 position/velocity를 None 으로 바꾸어 Isaac Lab이 PhysX position/velocity target buffer에 쓰지 못하게 한다. 최종적으로 PhysX에는 joint_efforts=tau_q 만 전달된다. 다음 physics step에는 Articulation이 원래 target buffer에서 새로운 ArticulationActions 를 만들기 때문에 velocity 입력은 다시 zero tensor이며 이전 출력의 None 이 target buffer에 저장되는 것은 아니다.

Actuator PD의 실제 수식과 output

현재 position-only action에서 실제 actuator PD는 다음과 같다.

```

tau_psi_requested = Kp(psi_target - psi) - Kd psi_dot
tau_psi_applied   = clip(tau_psi_requested, -tau_limit, +tau_limit)
tau_q_applied     = J-T tau_psi_applied

```

Custom actuator는 두 종류의 torque를 별도로 보관한다.

Buffer	의미
computed_actuator_effort	clip 전 actuator torque 요청값; torque-limit reward 가 saturation 요구를 검사할 때 사용
applied_actuator_effort	actuator torque limit으로 clip된 실제 actuator torque; torque L2 reward에 사용
computed_effort	clip 전 actuator torque를 joint coordinate로 변환한 값
applied_effort	clip된 actuator torque를 joint coordinate로 변환하여 PhysX에 전달하는 값

Torque와 velocity limit

Actuator와 joint limit은 역할이 다르다.

Limit	현재 설정	역할
Actuator torque limit	mechanical maximum의 0.9	tau_psi 실제 clip과 clip 전 초과 요청 reward 기준
Actuator velocity limit	mechanical maximum의 0.9	abs(psi_dot) 초과 reward 기준; state를 강제로 clip하지 않음

Limit	현재 설정	역할
PhysX joint effort limit	factor 없는 joint mechanical maximum	ADAPT 변환된 τ_q 에 대한 최종 solver 안전 상한

PhysX joint effort limit은 actuator torque에 0.9를 다시 곱하는 계산이 아니다. τ_q 가 상한 안에 있으면 그대로 적용하고 상한을 넘을 때만 PhysX가 clip한다.

Observation과 reward에서 ADAPT 사용

observations.py 는 현재 joint state를 같은 transmission으로 변환한다.

```
actuator_pos_rel = J^-1 (q - q_default)
actuator_vel_rel = J^-1 (q_dot - q_dot_default)
```

rewards.py 는 custom actuator buffer와 transmission을 사용한다.

```
actuator_torques_l2      -> applied_actuator_effort
actuator_torque_limits   -> computed_actuator_effort의 limit 초과량
actuator_vel_l2          -> J^-1 q_dot
actuator_velocity_limits -> J^-1 q_dot의 velocity limit 초과량
actuator_acc_l2          -> J^-1 q_ddot
joint_action_target_pos_limits -> mapped q_target의 joint soft-limit 초과량
```

한 policy step의 전체 요약

```
[100 Hz policy]
observation(q, q_dot -> psi, psi_dot)
-> network raw actuator action
-> clip/scale
-> psi_target
-> q_target
-> Articulation joint_pos_target buffer

[500 Hz physics, 위 target으로 5회 반복]
current q, q_dot + q_target + zero velocity/effort targets
-> RoK4AdaptActuator.compute()
-> psi, psi_dot, psi_target
-> actuator PD tau_psi
-> actuator torque clip
-> joint torque tau_q
-> PhysX joint effort
```

검증 포인트

- Action Manager 출력 dimension은 13이어야 한다.
- Policy observation은 5-step history를 포함해 240 dimension이어야 한다.
- actions.py 와 actuator가 같은 transmission instance를 사용해야 한다.
- control_action.joint_positions 는 runtime에서 None 이면 안 된다.
- 현재 joint_vel_target 과 joint_effort_target 은 zero tensor여야 한다.
- compute() 출력은 joint effort만 남기고 position/velocity target은 None 이어야 한다.
- Joint PhysX drive stiffness/damping은 사용하지 않고 explicit actuator torque가 적용되어야 한다.

- Joint PhysX effort limit은 100% mechanical maximum, actuator torque clip은 90%여야 한다.

Isaac Lab 원본과의 경계

이 구현은 Isaac Lab 원본 source를 수정하지 않는다. RoK4 로컬 class가 Isaac Lab public actuator/action interface를 상속하고, Isaac Lab Articulation 이 제공하는 target/state 전달 경로를 그대로 사용한다. 변경 대상은 모두 /home/rc1ab/rok4_lab 내부다.